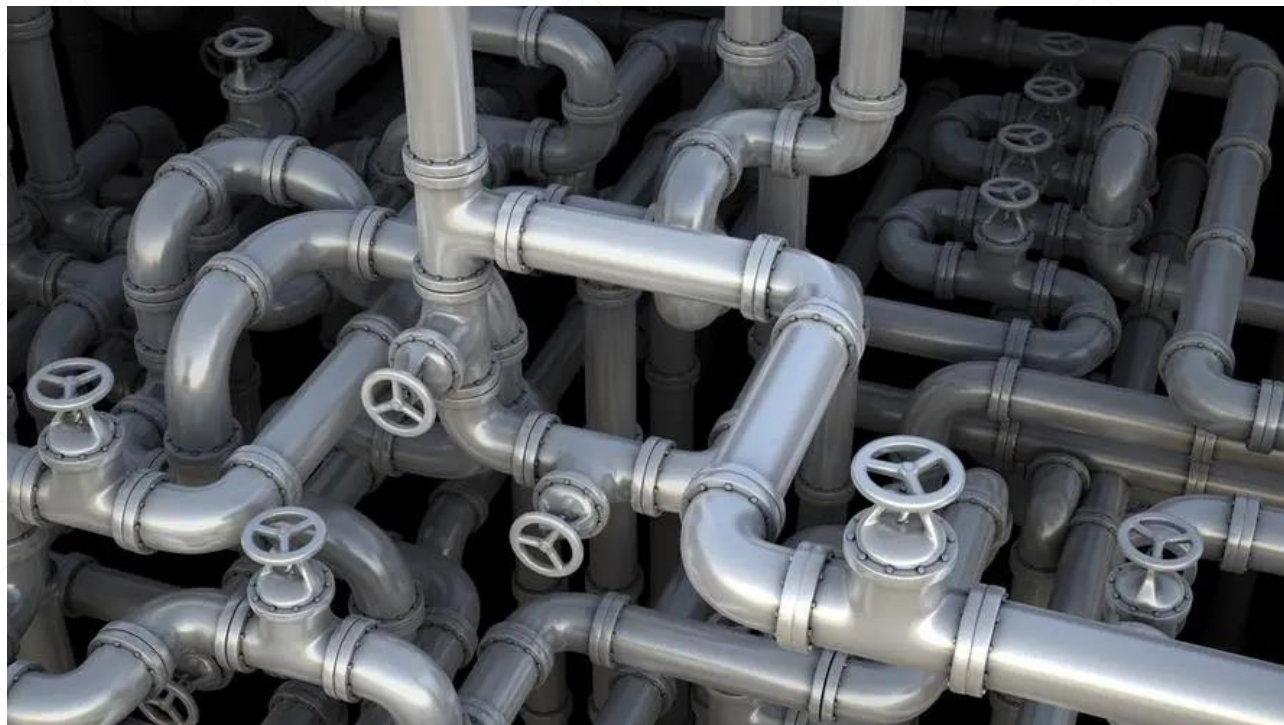


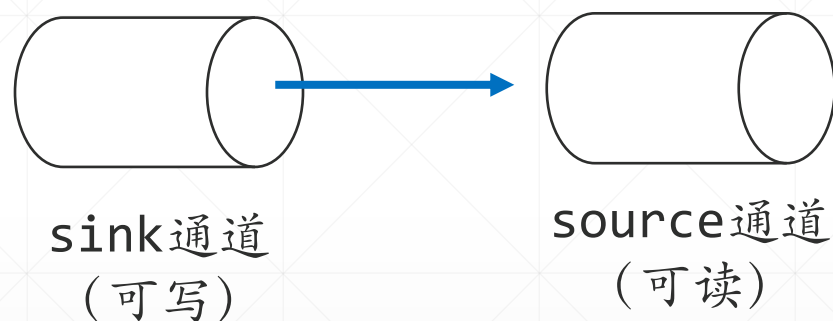


管道

北京理工大学计算机学院
金旭亮

管道

管道 (Pipe) 由一对通道所构成，一个通道发送数据，另一个通道接收数据。

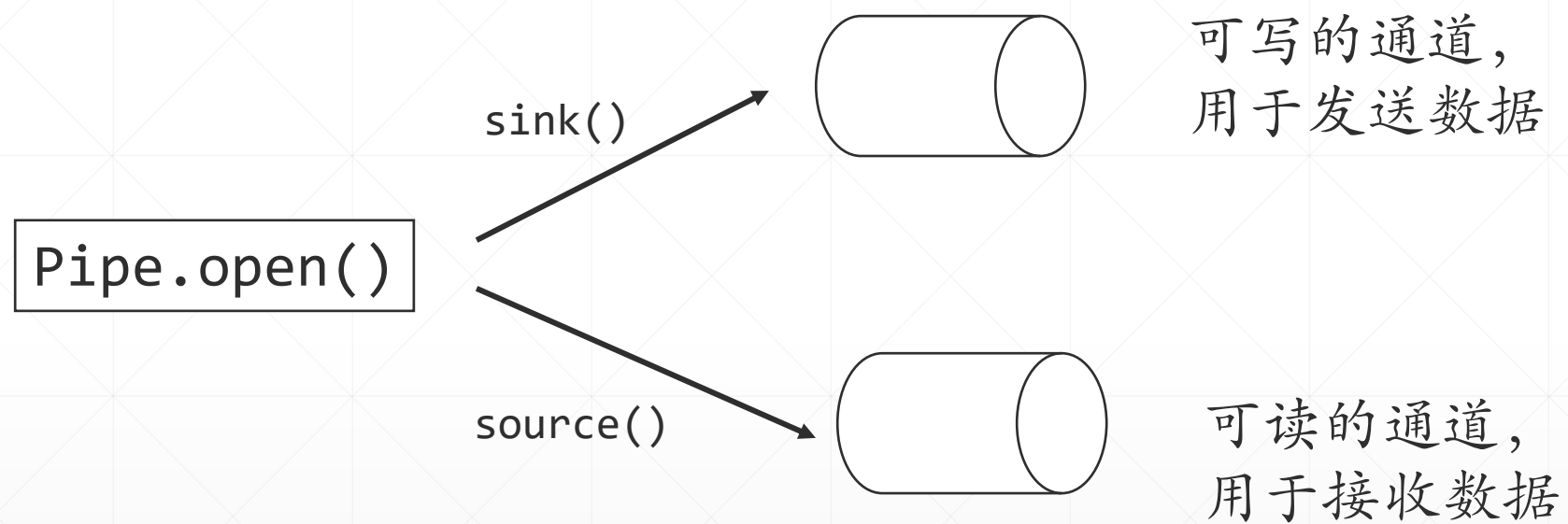


通道发送数据之后，将阻塞等待另一个通道接收数据。



管道可以在同一个JVM进程内部高效地传送数据，比较适合于“生产者” - “消费者”场景。

基本编程模型



Pipe示例

```
//使用Channel构建线程间的管道
private static void threadCommunicationViaChannelPipe()
    throws IOException {
    //构建管道
    Pipe pipe = Pipe.open();
    //数据发送方
    Runnable sender = () -> {...};
    //数据接收方
    Runnable receiver = () -> {...};

    new Thread(sender).start();
    new Thread(receiver).start();
}
```

数据发送

使用Buffer保存要发送的数据，使用SinkChannel将这些数据发送出去。

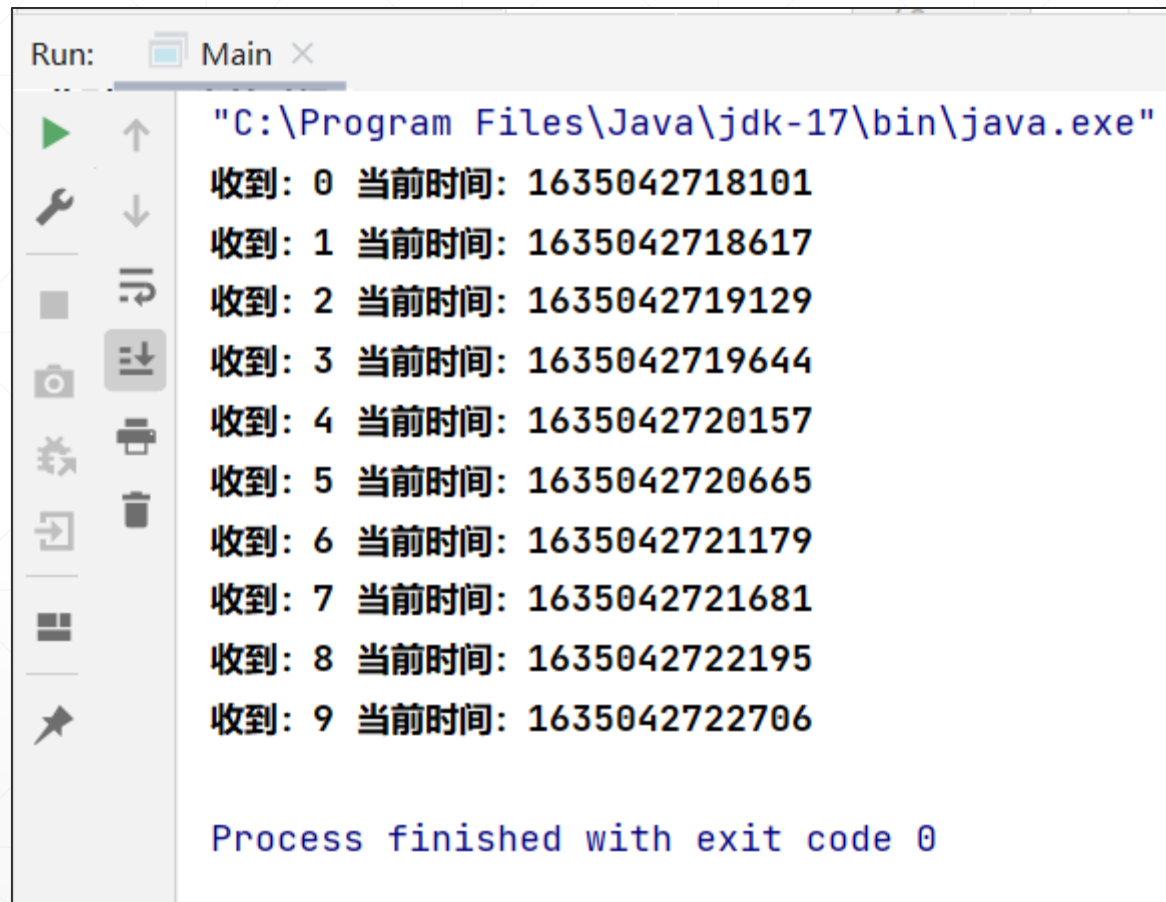
```
Runnable sender = () -> {  
    Pipe.SinkChannel sinkChannel = pipe.sink();  
    ByteBuffer buffer = ByteBuffer.allocate(256);  
    for (int i = 0; i < 10; i++) {  
        String currentTime = i + " 当前时间: " + System.currentTimeMillis();  
        buffer.put(currentTime.getBytes(StandardCharsets.UTF_8));  
        buffer.flip();  
        while (buffer.hasRemaining()) {  
            try {  
                sinkChannel.write(buffer);  
            } catch (IOException e) {  
                e.printStackTrace();  
            }  
        }  
        buffer.flip();  
        try {  
            Thread.sleep( millis: 500);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
};
```

数据接收

```
Runnable receiver = () -> {  
    Pipe.SourceChannel sourceChannel = pipe.source();  
    ByteBuffer buffer = ByteBuffer.allocate(256);  
    for (int i = 0; i < 10; i++) {  
        int bytesRead = 0;  
        try {  
            bytesRead = sourceChannel.read(buffer);  
            byte[] timeStringData = new byte[bytesRead];  
            buffer.flip();  
            buffer.get(timeStringData);  
            String timeString = new String(timeStringData, StandardCharsets.UTF_8);  
            System.out.println("收到: " + timeString);  
            buffer.flip();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
};
```

使用SourceChannel接收数据，保存到Buffer中。

运行截图



The screenshot shows a 'Run' console window from an IDE. The title bar indicates the process is 'Main'. The console output shows the execution of the Java command interpreter, followed by ten lines of received data and timestamps. The process concludes with an exit code of 0.

```
Run: Main x
"C:\Program Files\Java\jdk-17\bin\java.exe"
收到: 0 当前时间: 1635042718101
收到: 1 当前时间: 1635042718617
收到: 2 当前时间: 1635042719129
收到: 3 当前时间: 1635042719644
收到: 4 当前时间: 1635042720157
收到: 5 当前时间: 1635042720665
收到: 6 当前时间: 1635042721179
收到: 7 当前时间: 1635042721681
收到: 8 当前时间: 1635042722195
收到: 9 当前时间: 1635042722706

Process finished with exit code 0
```